

```
a[2] = b[2]*c[2];  
a[3] = b[3]*c[3];
```

This unrolled code has no loop overhead code at all, that is, no iteration variable and no tests. But the unrolled loop has the same problems as the inlined procedure—it may interfere with the cache and expands the amount of code required.

We do not, of course, have to fully unroll loops. Rather than unroll the above loop four times, we could unroll it twice. The following code results:

```
for (i = 0; i < 2; i++) {  
    a[i*2] = b[i*2]*c[i*2];  
    a[i*2 + 1] = b[i*2 + 1]*c[i*2 + 1];  
}
```

In this case, since all operations in the two lines of the loop body are independent, later stages of the compiler may be able to generate code that allows them to be executed efficiently on the CPU's pipeline.

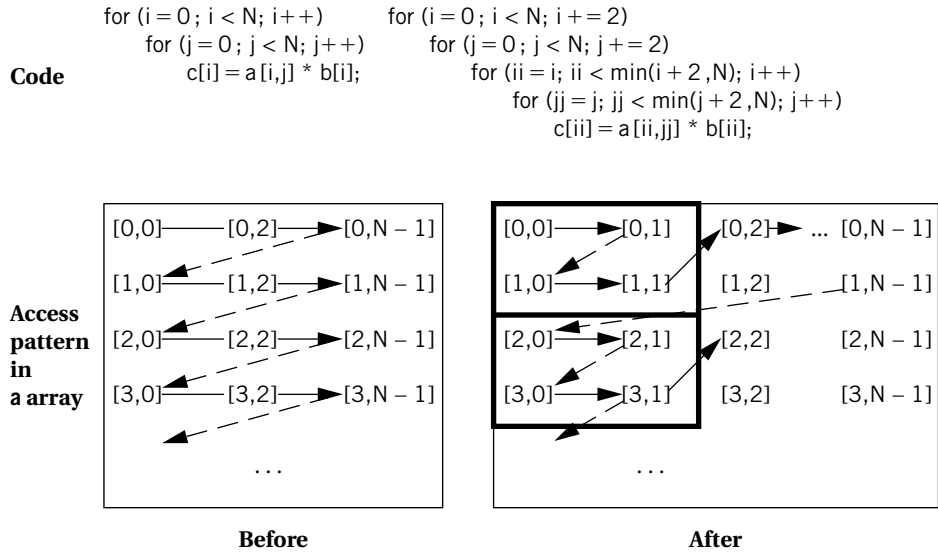
Loop fusion combines two or more loops into a single loop. For this transformation to be legal, two conditions must be satisfied. First, the loops must iterate over the same values. Second, the loop bodies must not have dependencies that would be violated if they are executed together—for example, if the second loop's i th iteration depends on the results of the $I + 1$ th iteration of the first loop, the two loops cannot be combined. **Loop distribution** is the opposite of loop fusion, that is, decomposing a single loop into multiple loops.

Loop tiling breaks up a loop into a set of nested loops, with each inner loop performing the operations on a subset of the data. An example is shown in Figure 5.17. Here, each loop is broken up into tiles of size two. Each loop is split into two loops—for example, the inner ii loop iterates within the tile and the outer i loop iterates across the tiles. The result is that the pattern of accesses across the a array is drastically different—rather than walking across one row in its entirety, the code walks through rows and columns following the tile structure. Loop tiling changes the order in which array elements are accessed, thereby allowing us to better control the behavior of the cache during loop execution.

We can also modify the arrays being indexed in loops. **Array padding** adds dummy data elements to a loop in order to change the layout of the array in the cache. Although these array locations will not be used, they do change how the useful array elements fall into cache lines. Judicious padding can in some cases significantly reduce the number of cache conflicts during loop execution.

5.5.5 Register Allocation

Register allocation is a very important compilation phase. Given a block of code, we want to choose assignments of variables (both declared and temporary) to

**FIGURE 5.17**

Loop tiling.

registers to minimize the total number of required registers. Example 5.5 illustrates the importance of proper register allocation.

Example 5.5

Register allocation

To keep the example small, we assume that we can use only four of the ARM's registers. In fact, such a restriction is not unthinkable—programming conventions can reserve certain registers for special purposes and significantly reduce the number of general-purpose registers available.

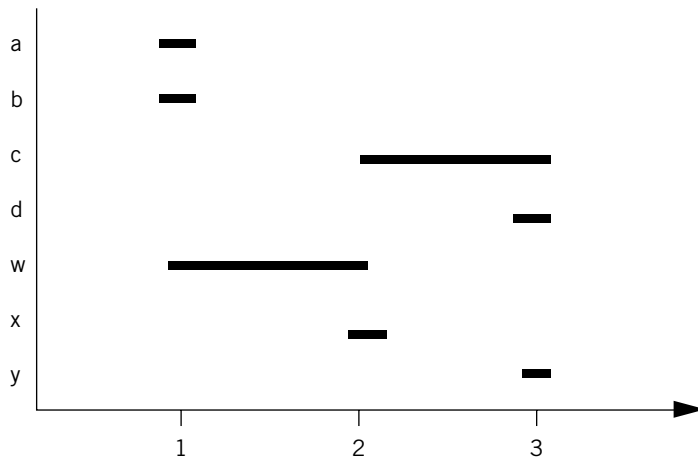
Consider the following C code:

```

w = a + b; /* statement 1 */
x = c + w; /* statement 2 */
y = c + d; /* statement 3 */

```

A naive register allocation, assigning each variable to a separate register, would require seven registers for the seven variables in the above code. However, we can do much better by reusing a register once the value stored in the register is no longer needed. To understand how to do this, we can draw a **lifetime graph** that shows the statements on which each statement is used. Appearing below is a lifetime graph in which the x-axis is the statement number in the C code and the y-axis shows the variables.



A horizontal line stretches from the first statement where the variable is used to the last use of the variable; a variable is said to be *live* during this interval. At each statement, we can determine every variable currently in use. The maximum number of variables in use at any statement determines the maximum number of registers required. In this case, statement two requires three registers: *c*, *w*, and *x*. This fits within the four registers limitation. By reusing registers once their current values are no longer needed, we can write code that requires no more than four registers. Appearing below is one register assignment.

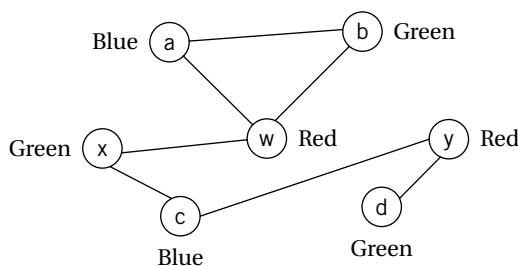
A	r0
B	r1
C	r2
D	r0
W	r3
X	r0
Y	r3

The ARM assembly code that uses the above register assignment follows:

```

LDR r0,[p_a]    ; load a into r0 using pointer to a (p_a)
LDR r1,[p_b]    ; load b into r1
ADD r3,r0,r1    ; compute a + b
STR r3,[p_w]    ; w = a + b
LDR r2,[p_c]    ; load c into r2
ADD r0,r2,r3    ; compute c + w, reusing r0 for x
STR r0,[p_x]    ; x = c + w
LDR r0,[p_d]    ; load d into r0
ADD r3,r2,r0    ; compute c + d, reusing r3 for y
STR r3,[p_y]    ; y = c + d

```

**FIGURE 5.18**

Using graph coloring to solve the problem of Example 5.5.

If a section of code requires more registers than are available, we must *spill* some of the values out to memory temporarily. After computing some values, we write the values to temporary memory locations, reuse those registers in other computations, and then reread the old values from the temporary locations to resume work. Spilling registers is problematic in several respects. For example, it requires extra CPU time and uses up both instruction and data memory. Putting effort into register allocation to avoid unnecessary register spills is worth your time.

We can solve register allocation problems by building a *conflict graph* and solving a graph coloring problem. As shown in Figure 5.18, each variable in the high-level language code is represented by a node. An edge is added between two nodes if they are both live at the same time. The graph coloring problem is to use the smallest number of distinct colors to color all the nodes such that no two nodes are directly connected by an edge of the same color. The figure shows a satisfying coloring that uses three colors. Graph coloring is NP-complete, but there are efficient heuristic algorithms that can give good results on typical register allocation problems.

Lifetime analysis assumes that we have already determined the order in which we will evaluate operations. In many cases, we have freedom in the order in which we do things. Consider the following expression:

$$(a + b) * (c - d)$$

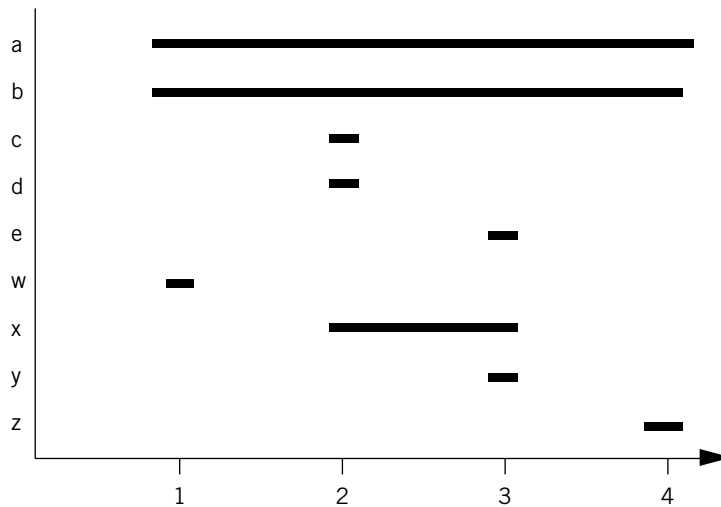
We have to do the multiplication last, but we can do either the addition or the subtraction first. Different orders of loads, stores, and arithmetic operations may also result in different execution times on pipelined machines. If we can keep values in registers without having to reread them from main memory, we can save execution time and reduce code size as well. Example 5.6 illustrates how proper *operator scheduling* can improve register allocation.

Example 5.6***Operator scheduling for register allocation***

Here is sample C code fragment:

```
w = a + b; /* statement 1 */
x = c + d; /* statement 2 */
y = x + e; /* statement 3 */
z = a - b; /* statement 4 */
```

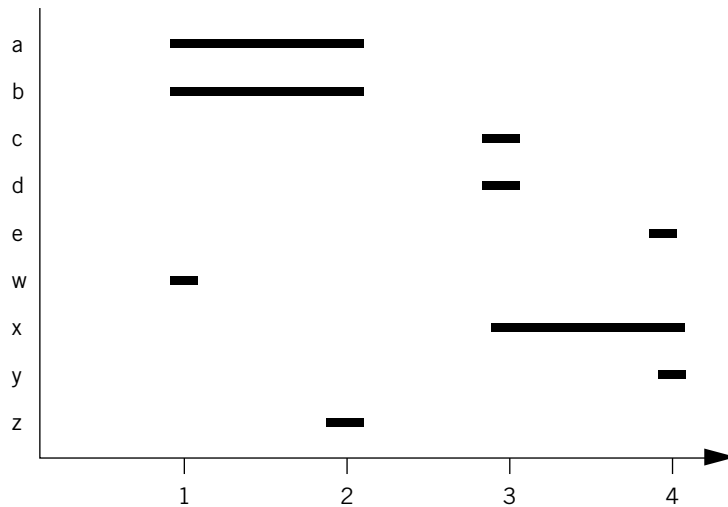
If we compile the statements in the order in which they were written, we get the register



Since *w* is needed until the last statement, we need five registers at statement 3, even though only three registers are needed for the statement at line 3. If we swap statements 3 and 4 (renumbering them 39 and 49), we reduce our requirements to three registers. The modified C code follows:

```
w = a + b; /* statement 1 */
z = a - b; /* statement 29 */
x = c + d; /* statement 39 */
y = x + e; /* statement 49 */
```

The lifetime graph for the new code appears below.



Compare the ARM assembly code for the two code fragments. We have written both assuming that we have only four free registers. In the *before* version, we do not have to write out any values, but we must read *a* and *b* twice. The *after* version allows us to retain all values in registers as long as we need them.

Before version

```
LDR r0,a
LDR r1,b
ADD r2,r0,r1
STR r2,w ; w = a + b
LDR r0,c
LDR r1,d
ADD r2,r0,r1
STR r2,x ; x = c + d
LDR r1,e
ADD r0,r1,r2
STR r0,y ; y = x + e
LDR r0,a ; reload a
LDR r1,b ; reload b
SUB r2,r1,r0
STR r2,z ; z = a - b
```

After version

```
LDR r0,a
LDR r1,b
ADD r2,r1,r0
STR r2,w ; w = a + b
SUB r2,r0,r1
STR r2,z ; z = a - b
LDR r0,c
LDR r1,d
ADD r2,r1,r0
STR r2,x ; x = c + d
LDR r1,e
ADD r0,r1,r2
STR r0,y ; y = x + e
```

5.5.6 Scheduling

We have some freedom to choose the order in which operations will be performed. We can use this to our advantage—for example, we may be able to improve the